

NAME:KRISHITHA G

REG NO:192524451

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO1_ASSESSMENT TOOL 1 _Application-Based Question

1 .Title : Intelligent Search Optimization System Using the Substitution Method

Aim

To solve the recurrence relation $T(n) = T(n/2) + \log n$ using the substitution method and determine its asymptotic complexity.

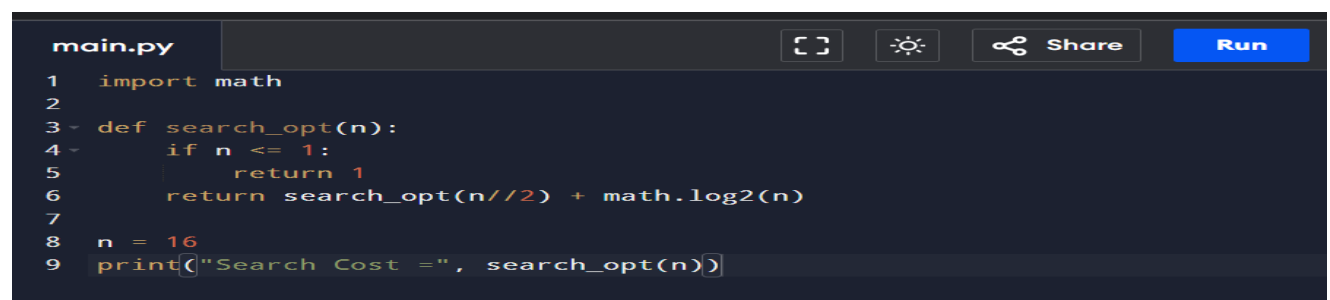
Software Used

- Python

Algorithm

- Start.
- Read the input size n .
- If $n \leq 1$, return the base value.
- Divide the problem into two equal halves ($n/2$).
- Perform logarithmic work ($\log n$) using indexed search.
- Recursively solve the subproblem of size $n/2$.
- Combine the recursive result with the logarithmic work.
- Display the total search cost.
- Stop.

Source Code (Python)



```
main.py [ ] [ ] [ ] Share Run
1 import math
2
3 def search_opt(n):
4     if n <= 1:
5         return 1
6     return search_opt(n//2) + math.log2(n)
7
8 n = 16
9 print("Search Cost =", search_opt(n))
```

OUTPUT

```
Output Clear  
Search Cost = 11.0  
  
=== Code Execution Successful ===
```

Analysis

Recurrence Relation

$$T(n) = T(n/2) + \log n$$

Expansion

$$\begin{aligned} T(n) &= T(n/2) + \log n \\ &= T(n/4) + \log(n/2) + \log n \\ &= T(n/8) + \log(n/4) + \log(n/2) + \log n \end{aligned}$$

After $k = \log n$ expansions,

$$\begin{aligned} T(n) &= T(1) + \sum_{i=0}^{\log n - 1} \log\left(\frac{n}{2^i}\right) \\ &= T(1) + \Theta((\log n)^2) \end{aligned}$$

Therefore,

$$\boxed{T(n) = \Theta((\log n)^2)}$$

Documentation

- The algorithm recursively divides the input into two halves.
- Each recursive level performs logarithmic work using indexed search operations.
- This makes the algorithm highly efficient for large datasets.
- Time Complexity: $\Theta((\log n)^2)$
- Space Complexity: $\Theta(\log n)$ (due to the recursion stack).

2.Title :

Aim

To solve the recurrence relation $T(n) = T(n/2) + n^2$ using the substitution method and analyze its time complexity.

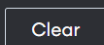
Software Used

- Python

Source Code

```
main.py    Share  Run  
1 def image_merge(n):  
2     if n <= 1:  
3         return 1  
4     return image_merge(n//2) + n*n  
5  
6 n = 8  
7 print("Merge Cost =", image_merge(n))
```

OUTPUT

```
Output   
Merge Cost = 85  
  
=== Code Execution Successful ===
```

Analysis

Recurrence Relation

$$T(n) = T(n/2) + n^2$$

Expansion

$$\begin{aligned} T(n) &= T(n/2) + n^2 \\ &= T(n/4) + \frac{n^2}{4} + n^2 \\ &= T(n/8) + \frac{n^2}{16} + \frac{n^2}{4} + n^2 \end{aligned}$$

After $\log n$ expansions,

$$T(n) = T(1) + n^2 \left(1 + \frac{1}{4} + \frac{1}{16} + \dots \right)$$

Since the series is geometric,

$$\boxed{T(n) = \Theta(n^2)}$$

Documentation

- The image dataset is recursively divided into two halves.
- Each merge operation performs **quadratic work** (n^2).
- Hence, the overall running time is dominated by the merge cost.
- **Time Complexity:** $\Theta(n^2)$.
- **Space Complexity:** $\Theta(\log n)$